Operating Systems

Assignment -10

Question 1 — Memory Management: Tracking Physical Memory Allocation

In xv6, physical memory is managed using the kernel page allocator (kalloc() and kfree()). Implement a mechanism to track the allocation and deallocation of physical memory pages.

Tasks:

a) Modify kalloc() and kfree() in kalloc.c to maintain counters for the total number of allocated and freed pages.

b) Implement a system call int memstats(void) that displays the total allocated pages, total freed pages, currently allocated pages, and available physical memory pages.

c) Write a user program memtest.c that allocates memory using sbrk(), displays memory statistics, releases the allocated memory, and displays the updated statistics.

Question 2 — Virtual Memory: Virtual-to-Physical Address Translation

In xv6, each process uses virtual addresses that are translated into physical addresses through page tables. Implement a system call to translate a given virtual address into its corresponding physical address.

Tasks:

a) Implement a system call int vtop(int virtual_address) that accepts a virtual address and uses walkpgdir() in vm.c to locate the corresponding page-table entry.

b) Extract the physical frame address and combine it with the page offset to calculate the complete physical address. Check whether the page-table entry is valid and present.

c) Write a user program vttest.c that allocates memory using sbrk(), passes different virtual addresses to vtop(), prints their corresponding physical addresses, and demonstrates the result for an invalid or unmapped virtual address.

Question 3 — Paging: Page Table Management and Page Protection

Paging divides virtual memory into fixed-size pages and physical memory into frames. Implement a mechanism in xv6 to inspect and modify page permissions.

Tasks:

a) Write a system call int pageinfo(int virtual_address) that checks whether the given virtual address is mapped and displays its physical frame number and page-table flags (P, W, U).

b) Implement a system call int setreadonly(int virtual_address) that removes write permission (PTE_W) from the corresponding page-table entry.

c) Write a user program pagetest.c that allocates a memory page, displays its page-table information, changes the page to read-only, and attempts to write to the protected page to demonstrate that a page fault occurs.

Question 4 — Page Replacement Algorithms: FIFO, LRU and Optimal

When the number of pages requested exceeds the available frames, the operating system must replace an existing page. Implement and compare three page replacement algorithms.

Tasks:

a) Write a user program pagereplace.c that implements FIFO (First-In First-Out), LRU (Least Recently Used), and Optimal Page Replacement. For every page reference, display the current frames and indicate whether it results in a page hit or page fault.

b) Test all three algorithms using the following reference string with 3 frames: **7 0 1 2 0 3 0 4 2 3 0 3 2 1 2 0 1 7 0 1**. Display the total number of page faults for each algorithm.

c) Run all three algorithms with 3, 4 and 5 frames using the same reference string. Prepare a comparison table showing the number of page faults, number of page hits, hit ratio, and page fault ratio. Explain which algorithm performs best for the given reference string.

Common Submission Requirements (for all 4 questions)

a) All modified xv6 kernel source files, including kalloc.c, vm.c, sysproc.c, syscall.c, syscall.h, defs.h, user.h and usys.S, as required. Clearly comment on every modification.

b) User test programs: memtest.c, vttest.c, pagetest.c and pagereplace.c.

c) Terminal output screenshots for each question demonstrating correct implementation and execution.

d) A short report (1–2 pages) explaining the implementation, modifications, algorithms used and observations from the output.